

Erratum to Hash Function Submission FEILIAN

FEILIAN Designer Team

October 2026

Note. The design of hash function FEILIAN does not change. All details should refer to the reference code that illustrates FEILIAN fully and correctly.

Original Document Version 1.0 (Released: June 2026)

Current Document Version 1.0.1 (Released: October 2026)

1 Corrections to Typos in Specification

The corrections to typos in specification.pdf are summarized in Table 1.

Table 1: Corrections to Specification

Location	Changes
Page 3, Section 2.1	The Initial Vector: $IV_7 = 0x3D84D5B5B5470917 \rightarrow IV_7 = 0x3F84D5B5B5470917;$
Page 9, Section 2.2.2	The Round Constant: $(0, tw, 0, \text{Const}) \rightarrow (0, \text{Const}, 0, tw);$
Page 40, Bibliography	Bib of “Second preimage attacks on dithered hash functions” [IKD+08] $\rightarrow$ [ABF+08]
Page 59-68, Appendix B.2/B.3	The printed test vectors for FEILIAN-768/512 wrong historical version $\rightarrow$ correct version

2 Corrections to Implementation

The corrections to software and hardware implementation are summarized in Table 2.

The Software implementation modifications are mainly refinements towards full-fledged version: (1) handle exceptional execution situation properly, (2) handle non-full-byte string properly, and (3) correct inconsistency typos in written comments.

3 Other modifications to Specification

We made few modification due to the community’s assessment report [1, 2] as below:

- Page 5, Section 2.2.1, we explicitly defined the counter t_0 as high word and t_1 as low word;
- Page 18, Section 4.2, we provided more detailed NIST SP-800-22 test results;
- Page 19, Section 4.3.1, we unified the notations in indifferentiability proof to be fully consistent with FEILIAN, during sketching and adapting the proof of HAIFA;
- Page 37, Section 6.2, we added a footnote to further emphasize the necessity of unambiguous encoding of context and counter;
- Page 37-38, Section 6.3, we added a note to illustrate that XOF should be regarded as an variable-output-length hash function, leveraging the same compression function FEILIAN- f .

Table 2: Corrections to Software and Hardware Implementation

Issues	Changes
Software Implementation	
Allocation Failure	pad_message() returns 0 if allocation fails. The internal hash function detects this and returns -1; CryptHash() propagates the error.
Non-full-byte string	After copying the input, pad_message() masks unused low-order bits in the final byte when <i>msg_bits</i> % 8 != 0. Those unused bits are cleared before compression.
Length Representability	Before converting the input length to size_t, CryptHash() checks whether it is representable. pad_message() calculates rounded-up byte and block counts using division and remainder, then checks num_blocks > SIZE_MAX / BLOCK_BYTES before calculating the allocation size.
Non-compliant usage	check whether the requested digest length matches the current version(512/768/1024) and returns an error if it does not.
Counter-word comments	hash_bits[0] holds the high word; hash_bits[1] holds the low word.
Hardware Implementation	
current block length omitted in 1SC/4SC/8SC RTL	revised the implementation of 1SC/4SC/8SC core to include the current block length in the compression input
Block/final flag/cumulative length be transaction-consistent	latch the message block, tweak (including the final flag and cumulative bit count) and initial chaining state, when a compression transaction is accepted, preserving them throughout compression
Counter-word	All RTL implementations use the same counter-word ordering: domain word 2 contains the upper 64 bits; and domain word 3 contains the lower 64 bits.
digest variants; parameter propagation	all four architectures support 512/768/1024 configurations. The top-level DIGEST_BITS parameter is forwarded to the core. It derives the version value and output-word count. Digest selection and MMIO address bounds now follow the configured output length.
Intermediate state exposure; clearing requirements for key-use	The 1SC/2SC/4SC cores restrict the digest output to completed hashes The top-level MMIO read path is also restricted to completed hashes
Readme typo	fixed the README to indicate: the four RTL architectures support three instances; the architecture names indicate SubColumn parallelism.

References

- [1] Mounir Idrassi. FEILIAN: technical assessment of the submitted specification and implementations, 2026.

- [2] Markku-Juhani O. Saarinen. ngcc.dev: Independent analysis of the NGCC candidates, 2026. Accessed 2026-10-08.